

Database Systems Lab

Python DB-API

Christian Rauch

(changed: August 25, 2025)

Python DB-API

The DB-API 2.0 provides a common framework for accessing SQL databases: Classes for connecting to databases, executing queries, fetching data, and handling transactions.

<https://peps.python.org/pep-0249/>

Different vendor provide concrete driver libraries:

SQLite → sqlite3 (built-in, no installation required)

PostgreSQL → psycopg2

MySQL → PyMySQL or mysql-connector-python

Oracle → cx_Oracle

Install the library (e.g., mysql-connector-python) using PIP.

```
1 $ python -m pip install mysql-connector-python
```

Connecting

Import the library.

```
1 import mysql.connector
2 import logging
```

Open a connection using credentials and other parameters.

```
1 try:
2     connection = mysql.connector.connect(
3         user='user', password='password',
4         host='localhost', database='database'
5     )
```

Use the connection to read or write data.

React to errors, usually by logging them.

```
1 except Error as e: logging.error(f"Error: {e}")
```

Close the connection after usage, typically in a finally section.

```
1 finally: connection.close()
```

Reading Data

Use the cursor to execute a SQL statement and process the result.

```
1 cursor = connection.cursor()
2
3 cursor.execute('SELECT * FROM rooms')
4
5 rows = cursor.fetchall() # or fetchmany(COUNT)
```

Convert SQL data types (e.g., CHAR, VARCHAR, FLOAT, DATE) to Python's native types (e.g., int, str, list) or user-defined types.

```
1 for tuple in rows:
2     room = str(tuple[0])
3     if tuple[1] is not None:
4         capacity = int(tuple[1])
5     ...
```

Mapping between SQL types and Python types requires custom code. ORM (Object-Relational Mapping) libraries like SQLAlchemy are an alternative.

Writing Data

Use the cursor to execute a SQL statement.

```
1 import mysql.connector
2 from mysql.connector import Error
3
4 try:
5     connection = mysql.connector.connect(...)
6     cursor = connection.cursor()
7
8     cursor.execute("INSERT INTO students (name,
9                     age, major) VALUES ('John Doe', 25, '
10                    Computer Science')")
11
12     connection.commit()
13
14 except Error as e:     print(f"Error: {e}")
15
16 finally:
17     if cursor:         cursor.close()
18     if connection:     connection.close()
```

Writing Binary Data

Use the cursor to execute a SQL statement to store binary data into a BLOB column.

```
1  ...
2
3      file_path = ...
4
5      # read the file in binary mode
6      with open(file_path, 'rb') as file:
7          binary_data = file.read()
8
9      cursor.execute("""
10         INSERT INTO files (filename, file_data)
11         VALUES (%s, %s)""", (... , binary_data))
12
13  ...
```

Predefined Parameterized Queries

SQL statements are usually predefined with parameter placeholders to prevent SQL injection attacks.

Placeholder in queries are substituted with supplied arguments during execution:

%s in MySQL and PostgreSQL,

? in SQLite

```
1 query = "INSERT INTO students (name, age, major)
2       VALUES (%s, %s, %s)"
3
4 data = ('John Doe', 25, 'Computer Science')
5
6 cursor.execute(query, data)
```

Transactions

Use transactions to ensure data consistency. Turn off auto-commit and handle errors properly.

```
1  try:
2      connection = mysql.connector.connect(...)
3      connection.autocommit = False
4      cursor = connection.cursor()
5
6      ... # Execute multiple related operations
7
8      connection.commit()
9
10 except Error as e:
11     connection.rollback()
12     logging.error(f"Transaction failed: {e}")
13
14 finally:
15     if cursor: cursor.close()
16     if connection: connection.close()
```

Transactions (Isolations)

Transaction isolation levels control how transactions interact with each other and what data they can see.

- ▶ **READ UNCOMMITTED:** Can read uncommitted changes from other transactions (dirty reads)
- ▶ **READ COMMITTED:** Only reads committed data, but may see different data on repeated reads
- ▶ **REPEATABLE READ:** Consistent reads within a transaction, but may see phantom rows
- ▶ **SERIALIZABLE:** Full isolation, transactions appear to run sequentially

Details differ in each library, check documentation.
SQLite only supports **SERIALIZABLE**.

1

```
connection.set_session(...)
```

2

```
connection.readonly = True
```

Conclusion

Thank you for your attention!